

Escopo do Projeto

App de Controle de Chamados Técnicos

Projeto Ionic/Angular sem banco de dados, utilizando arrays e services

1. Tema do Projeto

Desenvolver um aplicativo mobile utilizando Ionic/Angular para controle de chamados técnicos. O aplicativo deverá permitir o cadastro, listagem, acompanhamento e atualização de chamados técnicos, utilizando arrays para armazenar os dados temporariamente, sem uso de banco de dados.

2. Projeto

Desenvolvimento de um aplicativo mobile com Ionic/Angular, aplicando conceitos fundamentais de páginas, navegação, formulários, arrays, services e componentes visuais do Ionic.

- Criar páginas e rotas no Ionic/Angular;
- Montar formulários com ngModel;
- Manipular arrays em TypeScript;
- Compartilhar dados entre telas por meio de um service;
- Cadastrar, listar, excluir e atualizar informações;
- Construir uma interface organizada para uso mobile.

3. Regras Gerais

- O projeto deverá ser desenvolvido em Ionic/Angular.
- Não poderá utilizar banco de dados.
- Os dados deverão ser armazenados em arrays.
- Os dados deverão ser compartilhados entre páginas por meio de um service Angular.
- O aplicativo deverá possuir navegação entre telas.
- Os formulários deverão possuir validações simples.
- O layout deverá estar organizado e adequado para mobile.
- O código deverá estar organizado e comentado quando necessário.

4. Telas Obrigatórias

Tela de Login

Campos: usuário e senha. Se ambos estiverem preenchidos, o usuário poderá acessar o sistema. Caso contrário, deverá ser exibida uma mensagem de erro.

Tela de Menu Principal

Tela exibida após o login, contendo botões de navegação para cadastro de chamado, listagem, cadastro de técnico, resumo, sobre e sair.

Tela de Cadastro de Chamado

Formulário para registrar novo chamado técnico, com solicitante, setor, título, descrição, prioridade, data, técnico responsável e status.

Tela de Lista de Chamados

Listagem dos chamados cadastrados no array, exibindo título, solicitante, prioridade, status e técnico responsável.

Tela de Detalhes do Chamado

Exibe todas as informações do chamado selecionado e permite acessar a atualização de status.

Tela de Atualização de Status

Permite alterar o status do chamado e registrar uma observação do atendimento.

Tela de Cadastro de Técnico

Permite cadastrar técnicos responsáveis pelo atendimento, com nome, especialidade, contato e situação.

Tela de Lista de Técnicos

Exibe os técnicos cadastrados no array, permitindo visualizar informações básicas e excluir registros.

Tela de Resumo dos Chamados

Apresenta totais calculados com base no array de chamados, como quantidade por status e por prioridade.

Tela Sobre o Aplicativo

Apresenta nome do aplicativo, aluno ou grupo, disciplina, objetivo do projeto e tecnologias utilizadas.

5. Campos Sugeridos

Chamado

- ID
- Nome do solicitante
- Setor
- Título do chamado
- Descrição do problema
- Prioridade: Baixa, Média, Alta ou Urgente
- Data de abertura
- Técnico responsável
- Status: Aberto, Em atendimento, Concluído ou Cancelado

- Observação

Técnico

- ID
- Nome do técnico
- Especialidade: Hardware, Software, Rede, Impressora, Sistema interno ou Outros
- Contato: telefone ou e-mail
- Situação: Ativo ou Inativo

6. Estrutura dos Dados

Exemplo de objeto para o array de chamados:

```
{
  id: 1,
  solicitante: 'João Silva',
  setor: 'Financeiro',
  titulo: 'Computador não liga',
  descricao: 'O computador do setor financeiro não está ligando.',
  prioridade: 'Alta',
  dataAbertura: '2026-04-30',
  tecnico: 'Carlos Técnico',
  status: 'Aberto',
  observacao: ''
}
```

Exemplo de objeto para o array de técnicos:

```
{
  id: 1,
  nome: 'Carlos Técnico',
  especialidade: 'Hardware',
  contato: 'carlos@email.com',
  situacao: 'Ativo'
}
```

7. Service Obrigatório

O projeto deverá possuir um service para armazenar e compartilhar os dados entre as telas.

```
chamados: any[] = [];
tecnicos: any[] = [];

adicionarChamado(chamado: any) {
  this.chamados.push(chamado);
}

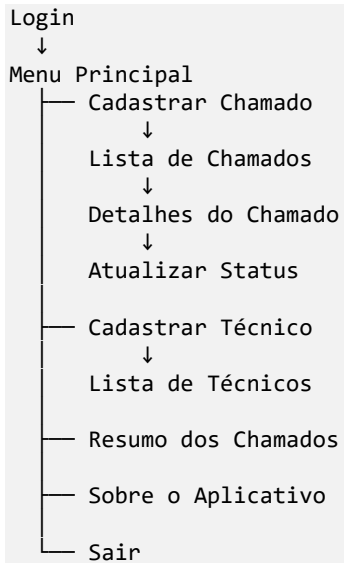
listarChamados() {
  return this.chamados;
}

excluirChamado(id: number) {
  this.chamados = this.chamados.filter(c => c.id !== id);
}

atualizarStatus(id: number, status: string, observacao: string) {
  const chamado = this.chamados.find(c => c.id === id);
  if (chamado) {
```

```
    chamado.status = status;
    chamado.observacao = observacao;
  }
}
```

8. Fluxo do Aplicativo



9. Validações Obrigatórias

Login

- Usuário obrigatório;
- Senha obrigatória.

Cadastro de Chamado

- Solicitante obrigatório;
- Título obrigatório;
- Descrição obrigatória;
- Prioridade obrigatória;
- Técnico obrigatório.

Cadastro de Técnico

- Nome obrigatório;
- Especialidade obrigatória;
- Contato obrigatório.

10. Entrega Esperada

- Projeto Ionic/Angular funcionando;

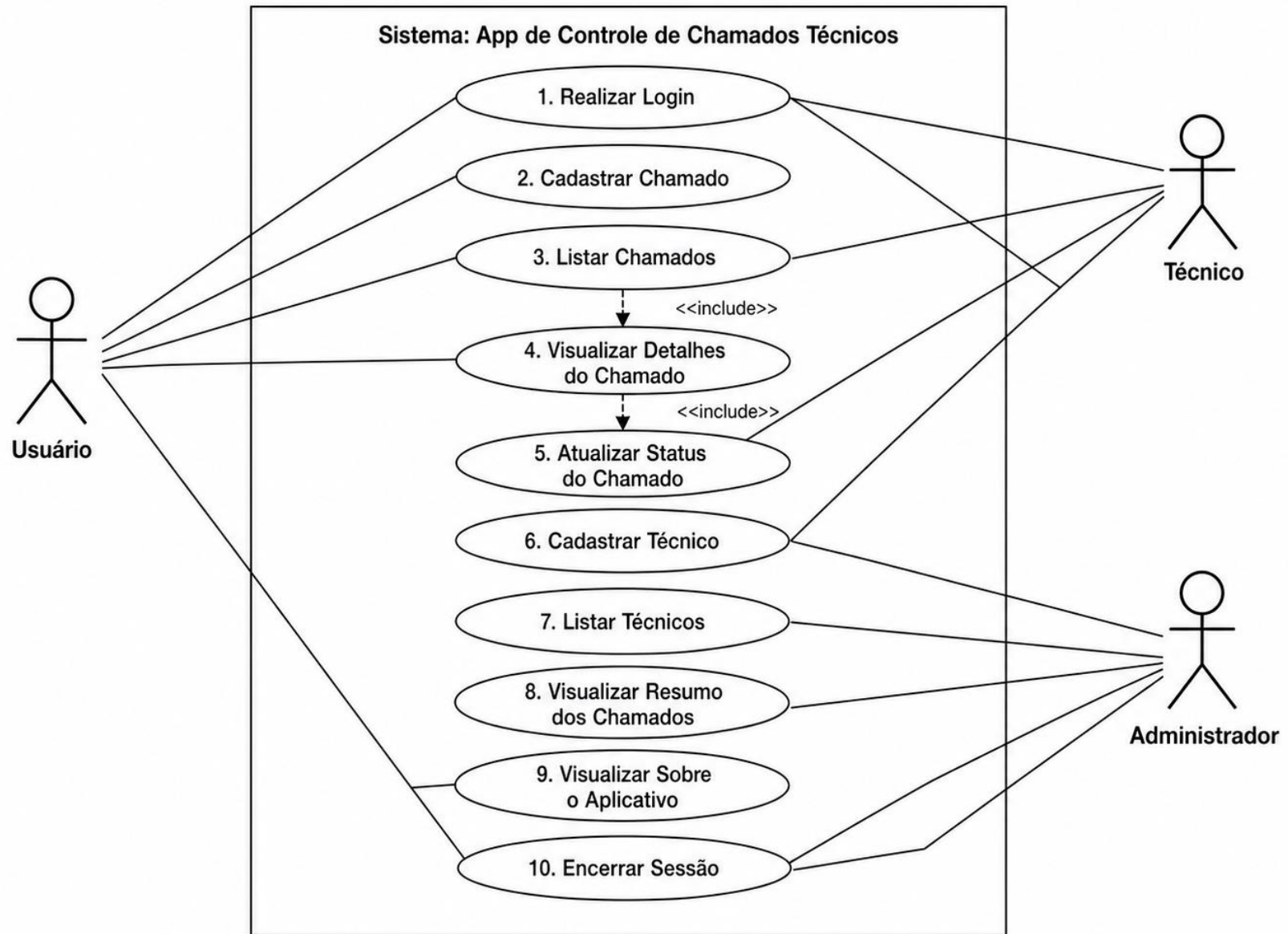
- Código organizado;
- Todas as telas solicitadas;
- Uso de arrays para armazenamento;
- Service para compartilhamento dos dados;
- Evidência de funcionamento, como prints das telas ou apresentação do aplicativo.

11. Observação Importante

Este projeto não deverá utilizar banco de dados. Os dados serão armazenados somente enquanto o aplicativo estiver em execução. Ao atualizar ou fechar o aplicativo, os dados poderão ser perdidos. Isso é esperado neste exercício, pois o objetivo é praticar arrays, formulários, navegação e services no Angular/Ionic.

Anexo 1 - Diagrama de Casos de Uso

Diagrama de Casos de Uso – App de Controle de Chamados Técnicos



Anexo 2 - Diagrama de Processos

Diagrama de Processos – App de Controle de Chamados Técnicos

